마이크로프로세서응용실습

결과보고서

홍정규 교수님

전자전기컴퓨터공학부

학번: 2019440100

이름: 이준헌

목차

1. 목표

2. 기능

3. 기능 구현

4. 동작 설명

5. 결론 및 고찰사항

1. 실험 목표

카메라를 이용하여 촬영한 이미지를 OpenCL을 통해 GPU로 가속하여 이미지를 처리하는 안드로이드 앱을 개발한다. 안드로이드 앱 기본 레이아웃 구성부터 시작하여 카메라 권한과 구동 등 안드로이드 개발 환경에 익숙해지고, OpenCL과 같은 GPGPU 프로그래밍 API을 경험한다.

2. 기능

2.1 카메라 프리뷰 및 홀드

안드로이드에서 카메라 권한을 받아 Surfaceview에서 Camera Preview를 구현하고, Capture 버튼을 통하여 Camera로 촬영된 이미지를 화면의 다른 Imageview에 홀드한다.

2.2 이미지 처리

CPU를 이용한 이미지 처리 뿐만 아니라 GPU를 이용한 가속 연산 또한 구현한다. 이때 이미지는 bmp 파일을 이용하여, 픽셀 단위 연산을 통해 Blur 및 Grayscale, 이미지 회전 등을 구현한다.

3. 기능 구현

3.1 카메라

우선 카메라 기능을 사용하기 위해서는 먼저 카메라 권한을 요청해야 한다. AndroidManifest.xml 파일에 카메라 권한을 추가해야 한다. 이후 런타임에서 권한을 요청해야 한다. 카메라를 어플리케이션에서 사용하기위해 Camera API를 이용한다.

카메라 프리뷰를 표시하기 위해서 Surfaceview를 추가하여 이 뷰를 이용하여 카메라 프리뷰를 표시한다. 버튼 객체를 이용하여 캡처를 구현하고 ImageView에 표시한다. 이후, 해당 이미지 파일을 이용하여 아래의 다양한 이미지 처리 연산을 수행하고 같은 ImageView에서 연산 후 이미지를 표시할 수 있게 한다.

3.2 CPU Blur

우선 입력으로 제공되는 bitmap 정보를 읽어와 넓이, 높이, 스트라이드 , 포맷 등을 확인한다. 이미지 처리 전 원본 픽셀 데이터를 복사하여 tempPixels 배열에 복사한다. 이후 가우시안 블러 마스크를 사용하여 각 픽셀에 블러 효과를 적용한다. 각 픽셀 주변의 픽셀 값을 가져와서 마스크의 가중치와 함께 가중 평균을 계산하여 새로운 픽셀 값으로 저장한다. 이미지 처리가 완료된 픽셀 데이터를 배열에 저장하고, 효과가 적용된 bitmap을 반환한다.

3.3 GPU Blur

안드로이드에서 GPU를 이용하여 가우시안 블러 효과를 구현한다. CPU를 이용한 이미지 처리와 마찬가지로 입력 bitmap을 받고, 원본 픽셀 데이터를 배열에 복사한다. OpenCL 커널 파일을 읽어오고, OpenCl을 사용하여 이미지에 가우시안 블러를 적용한다. 이후 처리

가 완료된 이미지 bitmap을 객체로 반환한다.

3.4 GPU Grayscale

3.3의 절차와 같으나, 이용하는 cl 파일을 가우시안 블러가 아닌 Grayscale cl 파일을 이용하여 흑백 이미지 처리를 하고 결과 이미지를 반환한다.

3.5 GPU Rotate

3.3의 절차와 같으나, 이용하는 cl 파일을 가우시안 블러가 아닌 Rotate cl 파일을 이용하여 회전 이미지 처리를 하고 결과 이미지를 반환한다.

4. 동작 설명

4.1 MainActivity.java

```
package com.example.termproject;

import
androidx.appcompat.app.AppCompatActivity;

import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.TextView;

public class MainActivity extends
AppCompatActivity {

    // 애플리케이션 시작 시 'native-lib'
라이브러리를 로드합니다.
    static {
        System.loadLibrary("OpenCLDriver");
    }
    // CPU 를 사용한 가우시안 블러 처리 함수
    public native Bitmap
GaussianBlurBitmap(Bitmap bitmap);
    // GPU 를 사용한 가우시안 블러 처리 함수
    public native Bitmap
GaussianBlurGPU(Bitmap bitmap);

    // GPU 를 사용한 그레이스케일 함수
    public native Bitmap
GaussianGreyGPU(Bitmap bitmap);

    // GPU 를 활용한 회전 함수
    public native Bitmap
GaussianRotateGPU(Bitmap bitmap);

    ImageView imgV;
    Bitmap buf_bitmap;

    @Override
    protected void onCreate(Bundle
savedInstanceState) {
        super.onCreate(savedInstanceState);

setContentView(R.layout.activity_main);

        final TextView tv = (TextView)
findViewById(R.id.editTextCalTime);
        imgV = (ImageView)
findViewById(R.id.imageView);
        Button btn = (Button)
findViewById(R.id.button);
        Button btn2 = (Button)
findViewById(R.id.button2);
        Button btn3 = (Button)
findViewById(R.id.button3);
        Button btn4 = (Button)
findViewById(R.id.button4);
        Button btn5 = (Button)
findViewById(R.id.button5);

        BitmapFactory.Options options = new
BitmapFactory.Options();
        options.inPreferredConfig =
Bitmap.Config.ARGB_8888;

        // 이미지 파일을 불러와 Bitmap 객체에
저장합니다.

buf_bitmap=BitmapFactory.decodeFile("/data/l
ocal/tmp/lena.bmp", options);

        tv.setText("Image Processing App");

        // 초기 이미지를 ImageView 에
설정합니다.
        imgV.setImageBitmap(buf_bitmap);

        // 첫 번째 버튼 클릭 이벤트 처리
        btn.setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                // 버튼 클릭 시, CPU 를
사용하여 이미지에 가우시안 블러 처리를 합니다.
                float start =
(float)System.nanoTime()/1000000;
```

```
                buf_bitmap =
GaussianBlurBitmap(buf_bitmap);
                float end =
(float)System.nanoTime()/1000000;
                float timesub = end-start;

imgV.setImageBitmap(buf_bitmap);
                // 처리 시간을 TextView 에
표시합니다.
                String ts =
Float.toString(timesub);
                tv.setText("Execution Time:
" + ts + "ms");
            }
        });

        // 두 번째 버튼 클릭 이벤트 처리
        btn2.setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                // 버튼 클릭 시, GPU 를
사용하여 이미지에 가우시안 블러 처리를 합니다.
                float start =
(float)System.nanoTime()/1000000;
                buf_bitmap =
GaussianBlurGPU(buf_bitmap);
                float end =
(float)System.nanoTime()/1000000;
                float timesub = end-start;

imgV.setImageBitmap(buf_bitmap);
                // 처리 시간을 TextView 에
표시합니다.
                String ts =
Float.toString(timesub);
                tv.setText("Execution Time:
" + ts + "ms");
            }
        });

        // 세 번째 버튼 클릭 이벤트 처리
        btn3.setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                // 버튼 클릭 시, 원본 이미지를
다시 불러옵니다.
                BitmapFactory.Options
options = new BitmapFactory.Options();
                options.inPreferredConfig =
Bitmap.Config.ARGB_8888;
                buf_bitmap =

BitmapFactory.decodeFile("/data/local/tmp/le
na.bmp",
                                options);

imgV.setImageBitmap(buf_bitmap);
                tv.setText("Time for
Calculation");
            }
        });

        btn4.setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                // 버튼 클릭 시, GPU 를
사용하여 이미지에 그레이 처리를 합니다.
                float start =
(float)System.nanoTime()/1000000;
                buf_bitmap =
GaussianGreyGPU(buf_bitmap);
                float end =
(float)System.nanoTime()/1000000;
                float timesub = end-start;

imgV.setImageBitmap(buf_bitmap);
                // 처리 시간을 TextView 에
표시합니다.
                String ts =
Float.toString(timesub);
                tv.setText("Execution Time:
" + ts + "ms");
            }
        });

        btn5.setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                // 버튼 클릭 시, GPU 를
사용하여 이미지에 회전 처리를 합니다.
                float start =
(float)System.nanoTime()/1000000;
                buf_bitmap =
GaussianRotateGPU(buf_bitmap);
                float end =
(float)System.nanoTime()/1000000;
                float timesub = end-start;

imgV.setImageBitmap(buf_bitmap);
                // 처리 시간을 TextView 에
표시합니다.
                String ts =
Float.toString(timesub);
                tv.setText("Execution Time:
" + ts + "ms");
            }
        });
    }

}
```

Bitmap 파일을 입력으로 받는 함수 4가지(각 CPU Blur, GPU Blur, Grayscale, Rotate) 선언 후, setOnClickListener을 통하여 총 5개의 버튼에 대한 이벤트를 정의한다. 버튼 1

번부터 순서대로, CPU Blru, GPU Blur, Original, Grayscale, Rotate이다.

4.2 OpenCLDriver.c

```
#include<jni.h>
#include<stdio.h>
#include<stdlib.h>
#include <sys/time.h>
#include <android/log.h>
#include <android/bitmap.h>

#include <CL/opencl.h>
#include <assert.h>

#define CL_FILE "/data/local/tmp/Blur.cl"
#define GREY_FILE "/data/local/tmp/Grey.cl"
#define ROTATE_FILE
"/data/local/tmp/Rotate.cl"

#define LOG_TAG "DEBUG"
#define LOGD(...)
__android_log_print(ANDROID_LOG_DEBUG,LOG_TA
G,__VA_ARGS__)
#define LOGE(...)
__android_log_print(ANDROID_LOG_ERROR,LOG_TA
G,__VA_ARGS__)

#define RGB8888_A(p) ((p & (0xff<<24))
>> 24 )
#define RGB8888_B(p) ((p & (0xff << 16)) >>
16 )
#define RGB8888_G(p) ((p & (0xff << 8))  >>
8 )
#define RGB8888_R(p) (p & (0xff) )

#define CHECK_CL(err) {\
    cl_int er = (err);\
    if(er<0 && er > -64){\
        LOGE("%d line, OpenCL
Error:%d\n",__LINE__,er);\
    }\
}



JNIEXPORT jobject JNICALL
Java_com_example_termproject_MainActivity_Ga
ussianBlurBitmap(JNIEnv *env, jobject thiz,
jobject bitmap)
{...}

JNIEXPORT jobject JNICALL
Java_com_example_termproject_MainActivity_Ga
ussianBlurGPU(JNIEnv *env, jobject thiz,

jobject bitmap)
{
    //getting bitmap info:
    LOGD("reading bitmap info...");
    AndroidBitmapInfo info;
    int ret;
    if ((ret = AndroidBitmap_getInfo(env,
bitmap, &info)) < 0) {
        LOGE("AndroidBitmap_getInfo()
failed ! error=%d", ret);
        return NULL;
    }
    LOGD("width:%d height:%d stride:%d",
info.width, info.height, info.stride);
    if (info.format !=
ANDROID_BITMAP_FORMAT_RGBA_8888) {
        LOGE("Bitmap format is not
RGBA_8888!");
        return NULL;
    }


    //read pixels of bitmap into native
memory :
    LOGD("reading bitmap pixels...");
    void* bitmapPixels;
    if ((ret = AndroidBitmap_lockPixels(env,
bitmap, &bitmapPixels)) < 0) {
        LOGE("AndroidBitmap_lockPixels()
failed ! error=%d", ret);
        return NULL;
    }
    uint32_t* src = (uint32_t*) bitmapPixels;
    uint32_t* tempPixels =
(uint32_t*)malloc(info.height *
info.width*4);
    int pixelsCount = info.height *
info.width;
    memcpy(tempPixels, src, sizeof(uint32_t)
* pixelsCount);

    ////GPU
    LOGD("GPU Start");
    FILE* file_handle;
    char* kernel_file_buffer, * file_log;;
    size_t kernel_file_size, log_size;

    unsigned char* cl_file_name = CL_FILE;
    unsigned char* kernel_name =
"kernel_blur";

    //Device input buffers
    cl_mem d_src;
    //Device output buffer
    cl_mem d_dst;

    cl_platform_id clPlatform;
//OpenCL platform
    cl_device_id device_id;
//device ID
    cl_context context;
//context
    cl_command_queue queue;
//command queue
```

```
    cl_program program;
//program
    cl_kernel kernel;
//kernel
    LOGD("cl_file_open");
    file_handle = fopen(cl_file_name, "r");
    if (file_handle == NULL) {
        printf("Couldn't find the file");
        exit(1);
    }

    //read kernel file
    fseek(file_handle, 0, SEEK_END);
    kernel_file_size = ftell(file_handle);
    rewind(file_handle);
    kernel_file_buffer =
(char*)malloc(kernel_file_size + 1);
    kernel_file_buffer[kernel_file_size] =
'\0';
    fread(kernel_file_buffer, sizeof(char),
kernel_file_size, file_handle);
    fclose(file_handle);
    LOGD("%s",kernel_file_buffer);
    LOGD("file_buffer_read");
    // Initialize vectors on host
    int i;

    size_t globalSize, localSize, grid;
    cl_int err;

    // Number of work items in each local
work group
    localSize = 64;
    int n_pix = info.width * info.height;

    //Number of total work items - localSize
must be devisor
    grid = (n_pix % localSize) ? (n_pix /
localSize) + 1 : n_pix / localSize;
    globalSize = grid * localSize;

    LOGD("calc grid and globalSize");
    //openCL 기반 실행

    //Bind to platform
    LOGD("error check");
    CHECK_CL(clGetPlatformIDs(1, &clPlatform,
NULL));
    LOGD("error end check");
    //Get ID for the device
    CHECK_CL(clGetDeviceIDs(clPlatform,
CL_DEVICE_TYPE_GPU, 1, &device_id, NULL));
    //Create a context
    context = clCreateContext(0, 1,
&device_id, NULL, NULL, &err);
    CHECK_CL(err);
    //Create a command queue
    queue = clCreateCommandQueue(context,
device_id, 0, &err);
    CHECK_CL(err);
    //Create the compute program from the
source buffer
    program =
clCreateProgramWithSource(context, 1, (const
char**)&kernel_file_buffer,
&kernel_file_size, &err);
    CHECK_CL(err);
    //Build the program executable
    err = clBuildProgram(program, 0, NULL,
NULL, NULL, NULL);
    LOGD("error 22 check");
    CHECK_CL(err);
    if (err != CL_SUCCESS) {
        //LOGD("%s", err);
        size_t len;
        char buffer[4096];
        LOGD("Error: Failed to build program
executable!");
        clGetProgramBuildInfo(program,
device_id, CL_PROGRAM_BUILD_LOG,
sizeof(buffer),
                                buffer, &len);

        LOGD("%s", buffer);
        //exit(1);
    }
    LOGD("error 323 check");

    //Create the compute kernel in the
program we wish to run
    kernel = clCreateKernel(program,
kernel_name, &err);
    CHECK_CL(err);



    //////openCL 커널 수행
    //Create the input and output arrays in
device memory for our calculation
    d_src = clCreateBuffer(context,
CL_MEM_READ_ONLY,
sizeof(uint32_t)*info.width*info.height,
NULL, NULL);
    d_dst = clCreateBuffer(context,
CL_MEM_WRITE_ONLY,
sizeof(uint32_t)*info.width*info.height,
NULL, NULL);

    //Write our data set into the input
array in device memory
    CHECK_CL(clEnqueueWriteBuffer(queue,
d_src, CL_TRUE, 0,
sizeof(uint32_t)*info.width*info.height,
tempPixels, 0, NULL, NULL));

    //Set the arguments to our compute
kernel
    CHECK_CL(clSetKernelArg(kernel, 0,
sizeof(cl_mem), &d_src));
    CHECK_CL(clSetKernelArg(kernel, 1,
sizeof(cl_mem), &d_dst));
    CHECK_CL(clSetKernelArg(kernel, 2,
sizeof(uint32_t), &info.width));
```

```
    CHECK_CL(clSetKernelArg(kernel, 3,
sizeof(uint32_t), &info.height));


    //Execute the kernel over the entire
range of the data set
    CHECK_CL(clEnqueueNDRangeKernel(queue,
kernel, 1, NULL, &globalSize, &localSize, 0,
NULL, NULL));
    //Wait for the command queue to get
serviced before reading back results
    CHECK_CL(clFinish(queue));
    //read the results form the device
    CHECK_CL(clEnqueueReadBuffer(queue,
d_dst, CL_TRUE, 0,
sizeof(uint32_t)*info.width*info.height, src,
0, NULL, NULL));


    // release OpenCL resources
    CHECK_CL(clReleaseMemObject(d_src));
    CHECK_CL(clReleaseMemObject(d_dst));
    CHECK_CL(clReleaseProgram(program));
    CHECK_CL(clReleaseKernel(kernel));
    CHECK_CL(clReleaseCommandQueue(queue));
    CHECK_CL(clReleaseContext(context));

    AndroidBitmap_unlockPixels(env, bitmap);
    //
    // free the native memory used to store
the pixels
    //
    free(tempPixels);
    return bitmap;

    return 0;
}

JNIEXPORT jobject JNICALL
Java_com_example_termproject_MainActivity_Ga
ussianGreyGPU(JNIEnv *env, jobject
thiz,jobject bitmap)
{...}

JNIEXPORT jobject JNICALL
Java_com_example_termproject_MainActivity_Ga
ussianRotateGPU(JNIEnv *env, jobject
thiz,jobject bitmap)
{...}
```

CPU Blur를 포함한 총 4가지의 이미지 처리 함수를 구현하였다. OpenCL을 이용하는 함수들의 경우, 사용하는 CL 파일의 종류만 다르고 그 외는 모두 일치한다.

4.3 CMakeList.txt

```
cmake_minimum_required(VERSION 3.22.1)


project("termproject")

add_library(native-lib
            SHARED
            native-lib.cpp)

add_library(GLES_mali STATIC IMPORTED)

add_library(OpenCLDriver
        SHARED
        OpenCLDriver.c)

find_library(log-lib
        log )

find_library( # Sets the name of the path
variable.
        jnigraphics-lib

target_link_libraries(OpenCLDriver

C:/OpenCL_lib_and_include/lib/libGLES_mali.s
o ${log-lib} ${jnigraphics-lib} )

target_include_directories(native-lib
PRIVATE src/main/cpp)
```

OpenCLDriver라는 공유 라이브러리를 정의하고 OpenCLDriver.c 파일을 해당 라이브러리에 포함시킨다. Target_link_libraries는 OpenCLDriver 라이브러리로 빌드 할 때 필요한 라이브러리를 링크한다.

4.4 activiy_main.xml

아래와 같이 linear layout 안에 5개의 버튼을 표시하고 그 아래 처리된 이미지가 나타날 ImageView를 구현하였다. 학번과 이름을 나타내는 TextView와 연산 시간을 나타내는 TextView는 ImageView 아래 위치하도록 하였다.

5. 결론 및 고찰사항

5.0 결론

해당 실험에서 카메라 프리뷰를 구현한 후, 실제 보드에서 프로그램을 작동시키면 실행하지 못하고 튕기는 현상이 반복되었다. 이는 이전 실험에서 안드로이드 카메라를 작동시킬 때, 카메라 권한이 제대로 설정되지 않았을 때 나타났던 문제 현상과 같아, 카메라에 대한 권한을 제대로 수정하고 실행하였으나 문제가 해결되지 않았다. 해당 권한 및 카메라 프리뷰 등을 다른 프로젝트 파일에서 구현하였을 때 문제없이 작동되었던 것으로 보아, 해당 프로그램을 구현할 때 기타 다른 원인들로 인해 제대로 실행되지 않았던 것으로 예상된다.

이로 인해서 최종적으로 해당 실험에서는 카메라 프리뷰를 비롯한 캡쳐와 해당 이미지 홀드 관련 기능과 레이아웃을 모두 제거하고, 저장된 이미지만을 사용하여 이미지 처리 기능을 작동시키도록 구현하였다.

5.1 고찰 사항

안드로이드 스튜디오와 같은 개발 환경을 해당 실험에서 처음 경험하여, 개발 작업이 지체된 점이 없지 않아 있다. 또한 안드로이드의 카메라 API 또한 복잡하여 하드웨어와 안드로이드 버전에 대한 호환성을 고려해야 하여 어려움을 겪었다.

OpenCL와 같은 GPGPU 프로그래밍은 지금까지 경험해본 개발 경험과 많이 달랐다. 특히 안드로이드에서 OpenCL을 사용한 것 또한 생소한 주제라서 인터넷에서 문서화된 자료가 없다시피 하여, 직접 부딪히며 배운 점이 많다.

이를 통해 OpenCL과 안드로이드 개발 환경에 대한 실질적인 경험을 얻었으며, 이에 대한 이해를 넓힐 수 있는 기회가 되었다. 또한, 현재는 안드로이드에서 GPGPU를 이용한 연산 프로그램이 생소할지라도, 향후에 디바이스 자체 성능의 발전과 함께 엣지 AI 등이 보편화되면, 안드로이드 환경에서의 GPGPU 프로그래밍 자체가 보편화 될 것으로 예상되어 더욱 더 배울 점이 많은 실험이라고 생각한다.

iiiiii

참고문헌

[i] 마이크로프로세서응용및실습 교안

KhronosGroup, Github, https://github.com/KhronosGroup/OpenCL-Guide

[iii] 안드로이드 디벨로퍼, Camera API, https://developer.android.com/guide/topics/media/camera?hl=ko#:~:text=,and%20videos%20in%20your%20applications